

Advanced Cryptography

Coursework 1 Discussion

TASK 1: PRIVATE AGGREGATION

The solution requires to work out that each client's status (0 for not updated and 1 for updated) needs to be secret shared via using a $(2, 3)$ secret sharing scheme. This means that each client secret shares their status using a degree one polynomial, producing three shares each. These shares are sent to the three servers, which simply add up shares from different clients. At the end, any two of the servers can reveal the overall sum using the public recombination vector for the resp. subset.

Any implementation for the Shamir secret sharing could be used, thus the only “new code” is to implement the addition of shares on each of the three servers, and the recombination, which is really a minimal extension to the formative coursework.

TASK 1: PRIVATE AGGREGATION, CONT.

The “write up” should have given some brief rationale/explanation for the system setup (unless this was explained by commentary in the code), plus the discussion of limitations. Limitations were in some sense specific to the implementation that was provided, so these could have been things like:

- ▶ Secure communication (encryption, or authenticated encryption) between client and server would need to be implemented.
- ▶ Authentication needs to be considered.
- ▶ Measures to protect against malicious adversaries would need to be implemented.
- ▶ Toy version of Shamir should be replaced by “proper version”.
- ▶ Toy version of random sampling needs to be replaced by “proper version”.
- ▶ Larger prime to accommodate more users.

TASK 1: MOST COMMON FLAWS

- ▶ Sharing the sum rather than the status. This is a catastrophic flaw, meaning the system is pointless.
- ▶ Sharing into a $(2,2)$ scheme. This is just not what the Task asked for.
- ▶ The statement that a security limitation is the small prime number. I.e. a powerful adversary could enumerate all polynomials and thereby recover the secret.

This is a fundamental misunderstanding of “perfect secrecy”. Perfect secrecy comes from the fact that an adversary learns no new information about a secret. I.e. even if an adversary can enumerate all possible secrets, they cannot tell (with any higher probability than before seeing shares/ciphertexts), which of them is the “correct” one. This does not depend on the size of the prime.

- ▶ Lack of any discussion of limitations, and/or lack of any “system design” (aka just supplying code).

TASK 2: DESIGN A SECRET SHARING SCHEME

The solution required to design a secret sharing scheme, whereby the following options were all seen as correct:

- ▶ The supreme leader either got or generated the secret. Thus we accepted that the supreme leader is the “dealer” or, that there is a separate “dealer”.
- ▶ Either of the groups (legion generals, knights) should be able to reconstruct, thus any (l, l) and separate (k, k) (with $k > l$) secret sharing schemes were acceptable. Thus we accepted both Boolean and Shamir versions.
- ▶ The term “either” implied that mixing individuals between groups was not o.k. so we wanted that the secret would be shared out via two separate secret sharing schemes.

Task 2 was graded by Sam, so any specific questions, please ask him.

TASK 2: MOST COMMON FLAWS

- ▶ A single secret sharing scheme that would enable a group consisting of both legion generals and knights to reconstruct the secret. This is a catastrophic flaw that leads to an incorrect system.
- ▶ Not explaining what the supreme leader gets/does, and how one could use this to control access to the weapon.
- ▶ No example, or only a partial example.

TASK 3: FIND THE SPY

The “shortcut” solution would have been to check out the Marvel series “Agents of Shield” to work out that Grant is the spy.

One can provide evidence for this claim by showing that combinations of 5 people excluding Grant give a consistent secret; whereas including Grant leads to various secrets.

The shared secret was 42, which is, according to the supercomputer “Deep Thought” in “A Hitchhiker’s Guide to the Galaxy”, the answer to the “Ultimate Question of Life, the Universe, and Everything”.

The single, most common flaw, was to randomly guess the spy, and then have no or a flawed explanation, for why they are the spy.

TASK M: AUDIT THE PYCRYPTODOME SSS IMPLEMENTATION

The task was to look out for some specifics (mentioned in the coursework) and then to find one potential problem in the code.

Most pinpointed as one potential problem, that their implementation actually doesn't seem to allow for a (n, n) scheme.

The single, most common flaw, was to not supply one potential problem.

TASK 4: DISTINGUISHABILITY

The task was about your understanding of security games= security definitions.

Task a. Presented you with an insecure game. Correct solutions consisted of either providing a single example of a strategy by which an adversary can distinguish L and R; or, to provide a correct general explanation of why L and R are distinguishable (i.e. they are distinguishable if c and n are not co-prime).

Task b. Asked you do actually provide a correct Left and Right oracle that expresses the indistinguishability of ciphertexts game. And then think about if the supplied encryption function would make L and R distinguishable. This was in part “book work” because there is just one such game which we talked about during lectures. The function was exactly the one from the L world in Task a, with $n = 10$ (i.e. it was “easy” to see that this was insecure for various inputs most notably 2 and 5).

TASK 4: MOST COMMON FLAWS

a.

- ▶ Just randomly guessing if or not the game was insecure and not having any reasoning.
- ▶ Reasoning based on the code of the games.
- ▶ Reasoning based on n begin odd vs even (close to what the problem is, but incorrect, consider e.g. $n = 2$).

b.

- ▶ Just copying the oracles from **a.**
- ▶ Producing two oracles with the same input m .
- ▶ Producing no oracle.

MOST SURPRISING MISTAKES/CHOICES

In Task 1, some people chose 21 as “prime number”.

In Task 3, some people attempted to do the brute force “by hand” despite having a working solution for Task 1.

Some people used rather complicated libraries of for Task 1 and then shared out the sum
...

Some people supplied a .tex file, or a .docx or no file at all. These clearly violated the requirements. In CW 2, I will give out 0 if you fail to include a pdf file in your submission.

SOME STATISTICS

Based on raw points (i.e. including hypothetical points of late submissions), the average was 21.52.

Out of 60 registered students, 39 received full points.

Points	Number of students	Predicted outcome
< 10	5	Fail
$13 \leq x \leq 16$	10	Borderline
$17 \leq x \leq 20$	6	Good pass
≥ 21	43	(Very) Good final grade

If you are among the students where I would predict a fail or borderline outcome, **please** make the most of Coursework 2.

Reminder: the formative coursework tasks (these can be discussed) are “mirrored” in the summative coursework tasks.